

基于 MindSpore 的 RNN 情感 分类实验手册



华为技术有限公司

版权所有 © 华为技术有限公司 2024。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址： 深圳市龙岗区坂田华为总部办公楼 邮编：518129

网址： <http://e.huawei.com>



目录

1 实验介绍	2
1.1 实验目的	2
1.2 实验清单	2
1.3 开发平台介绍	3
2 基于 MindSpore 的 RNN 情感分类	4
2.1 实验环境	4
2.2 数据准备	5
2.3 数据集预处理	10
2.4 模型构建	11
1.1.1 Embedding	11
2.4.2 RNN (循环神经网络)	11
2.4.3 Dense	13
2.4.4 损失函数与优化器	14
2.4.5 训练逻辑	14
2.4.6 评估指标和逻辑	15
2.5 模型训练与保存	16
2.6 模型加载与测试	17
2.7 自定义输入测试	17
2.8 实验小结	18

1 实验介绍

情感分类是自然语言处理中的经典任务，是典型的分类问题。本实验通过 MindSpore 实现一个基于 RNN 网络的情感分类模型，实现如下的效果：

```
输入: This film is terrible
正确标签: Negative
预测标签: Negative

输入: This film is great
正确标签: Positive
预测标签: Positive
```

图1-1 情感分类任务实现的效果

1.1 实验目的

本案例使用 MindSpore 框架进行深度学习网络 RNN 搭建，利用 IMDB 影评数据集进行模型训练、准确率验证，最后对英文的影评文本进行情感分类（Positive：积极；Negative：消极）。通过本实验可以了解到深度学习任务开发的具体流程，包括：数据收集、数据处理、模型搭建、模型训练及文本内容情感分类；了解经典循环神经网络 RNN 的结构特点和不同 RNN 模型的改进方式；以及深度学习自然语言处理任务中的重要概念，包括：词向量、词嵌入、时序模型、RNN 中 cell 单元的结构、LSTM 模型中的门控机制等。

1.2 实验清单

实验	简述	难度	软件环境	开发环境
基于MindSpore的RNN情感分类	本实验使用MindSpore框架搭建RNN网络，利用IMDB影评数据集进行模型训练和文本分类	中级	Python3.9 MindSpore2.2	ModelArts、Ascend

1.3 开发平台介绍

昇腾 (Ascend) 是华为自研的一款基于达芬奇架构的 AI 处理器，具有超高算力的和极高的能效比，最高可达 320TFLOPS (FP16) 的浮点计算能力。昇腾的片上系统 (SoC) 还集成了多个 CPU、DVPP 和任务调度器，因而具有自我管理的能力，可以充分发挥其高算力的特点。强大的矩阵、向量运行能力，在进行海量数据运算的神经网络训练场景有巨大的优势。

昇思 MindSpore (官方网站：<https://www.mindspore.cn/>) 是一种适用于端边云场景的新型开源深度学习训练/推理框架。MindSpore 提供了友好的设计和高效的执行，旨在提升数据科学家和算法工程师的开发体验，并为 Ascend AI 处理器提供原生支持，以及软硬件协同优化。同时，MindSpore 作为全球 AI 开源社区，致力于进一步开发和丰富 AI 软硬件应用生态。

- ModelArts (官方网站：<https://console.huaweicloud.com/modelarts/>) 是面向 AI 开发者的一站式开发平台，提供海量数据预处理及半自动化标注、大规模分布式训练、自动化模型生成及模型按需部署能力，帮助用户快速创建和部署 AI 应用，管理全周期 AI 工作流。

2 基于 MindSpore 的 RNN 情感分类

2.1 实验环境

在开始本实验前，需要完成实验环境搭建工作。

步骤 1 进入 ModelArts 开发环境

环境搭建方式可参考下方环境搭建手册中 “2.华为云 ModelArts 环境搭建（训练环境）” 部分内容。注意此实验因数据处理量比较大，创建环境时，磁盘规格自定义为 200GB。



ModelArts云环境
搭建指南.docx

步骤 2 打开 Notebook

打开 Notebook 控制台后，新建或打开 ipynb 文件，选择 MindSpore 环境作为 Kernel，即可开始编辑实验代码。

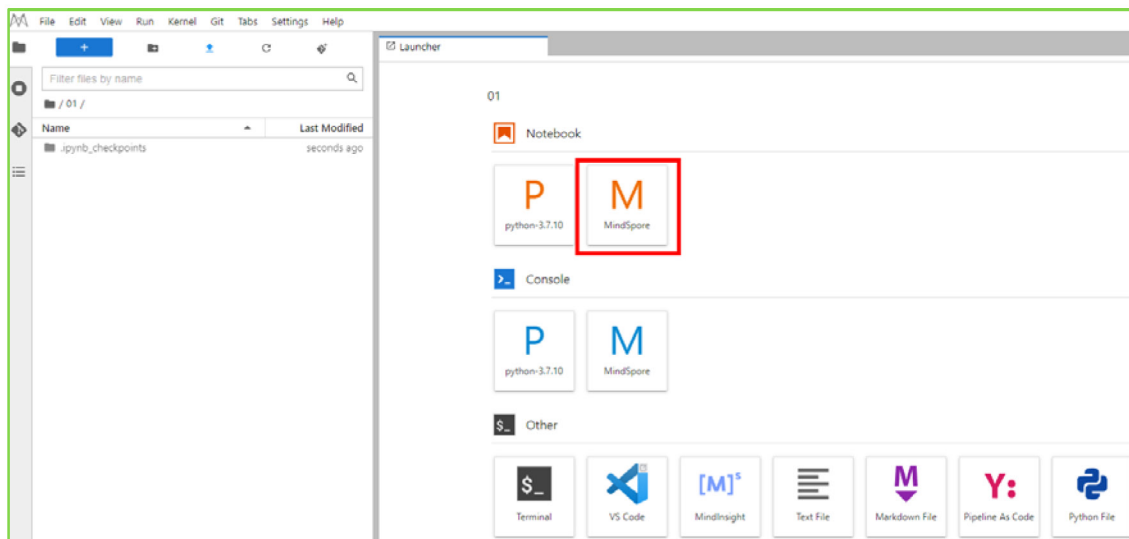


图2-1 创建 ipynb 文件

2.2 数据准备

本实验使用情感分类的经典数据集 [IMDB 影评数据集](#)，数据集包含 Positive 和 Negative 两类，下面为其样例：

Review	Label
"Quitting" may be as much about exiting a pre-ordained identity as about drug withdrawal. As a rural guy coming to Beijing, class and success must have struck this young artist face on as an appeal to separate from his roots and far surpass his peasant parents' acting success. Troubles arise, however, when the new man is too new, when it demands too big a departure from family, history, nature, and personal identity. The ensuing splits, and confusion between the imaginary and the real and the dissonance between the ordinary and the heroic are the stuff of a gut check on the one hand or a complete escape from self on the other.	Negative
This movie is amazing because the fact that the real people portray themselves and their real life experience and do such a good job it's like they're almost living the past over again. Jia Hongsheng plays himself an actor who quit everything except music and drugs struggling with depression and searching for the meaning of life while being angry at everyone especially the people who care for him most.	Positive

图2-2 IMDB 数据集内容示例

此外，需要使用预训练词向量对自然语言单词进行编码，以获取文本的语义特征，本节选取 [Glove](#) 词向量作为 Embedding。

步骤 1 导入依赖库

为了方便数据集和预训练词向量的下载，首先设计数据下载模块，实现可视化下载流程，并保存至指定路径。数据下载模块使用 requests 库进行 http 请求，并通过 tqdm 库对下载百分比进行可视化。此外针对下载安全性，使用 IO 的方式下载临时文件，而后保存至指定的路径并返回。

```
import os
import shutil
import requests
import tempfile
from tqdm import tqdm
from typing import IO
from pathlib import Path
from mindspore import context

context.set_context(mode=context.GRAPH_MODE, device_target="Ascend", device_id=0)

# 指定保存路径为 `home_path/.mindspore_examples`
cache_dir = Path.home() / '.mindspore_examples'

def http_get(url: str, temp_file: IO):
    """使用 requests 库下载数据，并使用 tqdm 库进行流程可视化"""
```

```
req = requests.get(url, stream=True)
content_length = req.headers.get('Content-Length')
total = int(content_length) if content_length is not None else None
progress = tqdm(unit='B', total=total)
for chunk in req.iter_content(chunk_size=1024):
    if chunk:
        progress.update(len(chunk))
        temp_file.write(chunk)
progress.close()

def download(file_name: str, url: str):
    """下载数据并保存为指定名称"""
    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir)
    cache_path = os.path.join(cache_dir, file_name)
    cache_exist = os.path.exists(cache_path)
    if not cache_exist:
        with tempfile.NamedTemporaryFile() as temp_file:
            http_get(url, temp_file)
            temp_file.flush()
            temp_file.seek(0)
            with open(cache_path, 'wb') as cache_file:
                shutil.copyfileobj(temp_file, cache_file)
    return cache_path
```

完成数据下载模块后，下载 IMDB 数据集进行测试(此处使用华为云的镜像用于提升下载速度)。下载过程及保存的路径如下：

```
#设置 no_proxy 环境变量
%env no_proxy='a.test.com,127.0.0.1,2.2.2.2'
imdb_path = download('aclImdb_v1.tar.gz', 'https://mindspore-
website.obs.myhuaweicloud.com/notebook/datasets/aclImdb_v1.tar.gz')
imdb_path
```

步骤 2 加载 IMDB 数据集

下载好的 IMDB 数据集为 tar.gz 文件，我们使用 Python 的 tarfile 库对其进行读取，并将所有数据和标签分别进行存放。原始的 IMDB 数据集解压目录如下：

```
├── aclImdb
│   ├── imdbEr.txt
│   ├── imdb.vocab
│   ├── README
│   ├── test
│   └── train
│       ├── neg
│       └── pos
```

数据集已分割为 train 和 test 两部分，且每部分包含 neg 和 pos 两个分类的文件夹，因此需分别 train 和 test 进行读取并处理数据和标签。

```
import re
import six
import string
import tarfile

class IMDBData():
    """IMDB 数据集加载器

    加载 IMDB 数据集并处理为一个 Python 迭代对象。

    """
    label_map = {
        "pos": 1,
        "neg": 0
    }
    def __init__(self, path, mode="train"):
        self.mode = mode
        self.path = path
        self.docs, self.labels = [], []

        self._load("pos")
        self._load("neg")

    def _load(self, label):
        pattern = re.compile(r"aclImdb/{}/{}/.*\.txt$".format(self.mode, label))
        # 将数据加载至内存
        with tarfile.open(self.path) as tarf:
            tf = tarf.next()
            while tf is not None:
                if bool(pattern.match(tf.name)):
                    # 对文本进行分词、去除标点和特殊字符、小写处理
                    self.docs.append(str(tarf.extractfile(tf).read()).rstrip(six.b("\n\r"))
                                     .translate(None, six.b(string.punctuation)).lower()).split())
                    self.labels.append([self.label_map[label]])
                tf = tarf.next()

    def __getitem__(self, idx):
        return self.docs[idx], self.labels[idx]

    def __len__(self):
        return len(self.docs)
```

完成 IMDB 数据加载器后，加载训练数据集进行测试，输出数据集数量：

```
imdb_train = IMDBData(imdb_path, 'train')
```

```
len(imdb_train)
```

输出信息如下：

```
25000
```

将 IMDB 数据集加载至内存并构造为迭代对象后，可以使用 mindspore.dataset 提供的 GeneratorDataset 接口加载数据集迭代对象，并进行下一步的数据处理，下面封装一个函数将 train 和 test 分别使用 GeneratorDataset 进行加载，并指定数据集中文本和标签的 column_name 分别为 text 和 label:

```
import mindspore.dataset as ds
def load_imdb(imdb_path):
    imdb_train = dataset.GeneratorDataset(IMDBData(imdb_path, "train"), column_names=["text", "label"],
shuffle=True)
    imdb_test = dataset.GeneratorDataset(IMDBData(imdb_path, "test"), column_names=["text", "label"], shuffle=
False)
    return imdb_train, imdb_test
```

加载 IMDB 数据集，可以看到 imdb_train 是一个 GeneratorDataset 对象。

```
imdb_train, imdb_test = load_imdb(imdb_path)
imdb_train
```

输出如下信息：

```
<mindspore.dataset.engine.datasets_user_defined.GeneratorDataset at 0x7f3885733710>
```

步骤 3 加载预训练词向量

预训练词向量是对输入单词的数值化表示，通过 nn.Embedding 层，采用查表的方式，输入单词对应词表中的 index，获得对应的表达向量。因此进行模型构造前，需要将 Embedding 层所需的词向量和词表进行构造。这里我们使用 Glove(Global Vectors for Word Representation)这种经典的预训练词向量，其数据格式如下：

Word	Vector
the	0.418 0.24968 -0.41242 0.1217 0.34527 -0.044457 -0.49688 -0.17862 -0.00066023 ...
,	0.013441 0.23682 -0.16899 0.40951 0.63812 0.47709 -0.42852 -0.55641 -0.364 ...

图2-3 预训练词向量格式

我们直接使用第一列的单词作为词表，使用 dataset.text.Vocab 将其按顺序加载；同时读取每一行的 Vector 并转为 numpy.array，用于 nn.Embedding 加载权重使用。具体实现如下：

```
import zipfile
import numpy as np

def load_glove(glove_path):
    glove_100d_path = os.path.join(cache_dir, 'glove.6B.100d.txt')
    if not os.path.exists(glove_100d_path):
        glove_zip = zipfile.ZipFile(glove_path)
```

```
glove_zip.extractall(cache_dir)

embeddings = []
tokens = []
with open(glove_100d_path, encoding='utf-8') as gf:
    for glove in gf:
        word, embedding = glove.split(maxsplit=1)
        tokens.append(word)
        embeddings.append(np.fromstring(embedding, dtype=np.float32, sep=' '))

# 添加 <unk>, <pad> 两个特殊占位符对应的 embedding
embeddings.append(np.random.rand(100))
embeddings.append(np.zeros((100,), np.float32))

vocab = dataset.text.Vocab.from_list(tokens, special_tokens=["<unk>", "<pad>"], special_first=False)
embeddings = np.array(embeddings).astype(np.float32)
return vocab, embeddings
```

由于数据集中可能存在词表没有覆盖的单词，因此需要加入<unk>标记符；同时由于输入长度的不一致，在打包为一个 batch 时需要将短的文本进行填充，因此需要加入<pad>标记符。完成后的词表长度为原词表长度+2。

加载 Glove 生成词表和词向量权重矩阵。

```
glove_path = download('glove.6B.zip', 'https://mindspore-
website.obs.myhuaweicloud.com/notebook/datasets/glove.6B.zip')
vocab, embeddings = load_glove(glove_path)
len(vocab.vocab())
```

输出信息如下：

```
400002
```

使用词表将 the 转换为 index id，并查询词向量矩阵对应的词向量：

```
idx = vocab.tokens_to_ids('the')
embedding = embeddings[idx]
idx, embedding
```

输出信息如下：

```
(0,
 array([-0.038194, -0.24487,  0.72812, -0.39961,  0.083172,  0.043953,
        -0.39141,  0.3344 , -0.57545,  0.087459,  0.28787, -0.06731,
         0.30906, -0.26384, -0.13231, -0.20757,  0.33395, -0.33848,
        -0.31743, -0.48336,  0.1464 , -0.37304,  0.34577,  0.052041,
         0.44946, -0.46971,  0.02628, -0.54155, -0.15518, -0.14107,
        -0.039722,  0.28277,  0.14393,  0.23464, -0.31021,  0.086173,
         0.20397,  0.52624,  0.17164, -0.082378, -0.71787, -0.41531,
         0.20335, -0.12763,  0.41367,  0.55187,  0.57908, -0.33477,
        -0.36559, -0.54857, -0.062892,  0.26584,  0.30205,  0.99775,
        -0.80481, -3.0243 ,  0.01254, -0.36942,  2.2167 ,  0.72201,
        -0.24978,  0.92136,  0.034514,  0.46745,  1.1079 , -0.19358,
```



```
-0.074575, 0.23353, -0.052062, -0.22044, 0.057162, -0.15806,
-0.30798, -0.41625, 0.37972, 0.15006, -0.53212, -0.2055,
-1.2526, 0.071624, 0.70565, 0.49744, -0.42063, 0.26148,
-1.538, -0.30223, -0.073438, -0.28312, 0.37104, -0.25217,
0.016215, -0.017099, -0.38984, 0.87424, -0.72569, -0.51058,
-0.52028, -0.1459, 0.8278, 0.27062 ], dtype=float32))
```

2.3 数据集预处理

通过加载器加载的 IMDB 数据集进行了分词处理，但不满足构造训练数据的需要，因此要对其进行额外的预处理。其中包含的预处理如下：

- 通过 Vocab 将所有的 Token 处理为 index id。
- 将文本序列统一长度，不足的使用<pad>补齐，超出的进行截断。

这里我们使用 mindspore.dataset 中提供的接口进行预处理操作。这里使用到的接口均为 MindSpore 的高性能数据引擎设计，每个接口对应操作视作数据流水线的一部分，详情请参考 MindSpore 数据引擎。首先针对 token 到 index id 的查表操作，使用 text.Lookup 接口，将前文构造的词表加载，并指定 unknown_token。其次为文本序列统一长度操作，使用 PadEnd 接口，此接口定义最大长度和补齐值(pad_value)，这里我们取最大长度为 500，填充值对应词表中<pad>的 index id。除了对数据集中 text 进行预处理外，由于后续模型训练的需要，要将 label 数据转为 float32 格式。

```
import mindspore as ms

lookup_op = dataset.text.Lookup(vocab, unknown_token='<unk>')
pad_op = dataset.transforms.c_transforms.PadEnd([500], pad_value=vocab.tokens_to_ids('<pad>'))
type_cast_op = dataset.transforms.c_transforms.TypeCast(mindspore.float32)
```

完成预处理操作后，需将其加入到数据集处理流水线中，使用 map 接口对指定的 column 添加操作。

```
imdb_train = imdb_train.map(operations=[lookup_op, pad_op], input_columns=['text'])
imdb_train = imdb_train.map(operations=[type_cast_op], input_columns=['label'])

imdb_test = imdb_test.map(operations=[lookup_op, pad_op], input_columns=['text'])
imdb_test = imdb_test.map(operations=[type_cast_op], input_columns=['label'])
```

由于 IMDB 数据集本身不包含验证集，我们手动将其分割为训练和验证两部分，比例取 0.7, 0.3。

```
imdb_train, imdb_valid = imdb_train.split([0.7, 0.3])
```

最后指定数据集的 batch 大小，通过 batch 接口指定，并设置是否丢弃无法被 batch size 整除的剩余数据。

```
imdb_train = imdb_train.batch(64, drop_remainder=True)
```



```
imdb_valid = imdb_valid.batch(64, drop_remainder=True)
```

2.4 模型构建

完成数据集的处理后，我们设计用于情感分类的模型结构。首先需要将输入文本(即序列化后的 index id 列表)通过查表转为向量化表示，此时需要使用 nn.Embedding 层加载 Glove 词向量；然后使用 RNN 循环神经网络做特征提取；最后将 RNN 连接至一个全连接层，即 nn.Dense，将特征转化为与分类数量相同的 size，用于后续进行模型优化训练。整体模型结构如下：

```
nn.Embedding -> nn.RNN -> nn.Dense
```

图2-4 模型结构

这里我们使用能够一定程度规避 RNN 梯度消失问题的变种 LSTM(Long short term memory)做特征提取层。下面对模型进行详解：

1.1.1 Embedding

Embedding 层又可称为 EmbeddingLookup 层，其作用是使用 index id 对权重矩阵对应 id 的向量进行查找，当输入为一个由 index id 组成的序列时，则查找并返回一个相同长度的矩阵，例如：

```
embedding = nn.Embedding(1000, 100) # 词表大小(index的取值范围)为1000，表示向量的size为100
input shape: (1, 16)                # 序列长度为16
output shape: (1, 16, 100)
```

图2-5 词向量接口使用示例

这里我们使用前文处理好的 Glove 词向量矩阵，设置 nn.Embedding 的 embedding_table 为预训练词向量矩阵。对应的 vocab_size 为词表大小 400002，embedding_size 为选用的 glove.6B.100d 向量大小，即 100。

2.4.2 RNN（循环神经网络）

循环神经网络（Recurrent Neural Network, RNN）是一类以序列（sequence）数据为输入，在序列的演进方向进行递归（recursion）且所有节点（循环单元）按链式连接的神经网络。下图为 RNN 的一般结构：

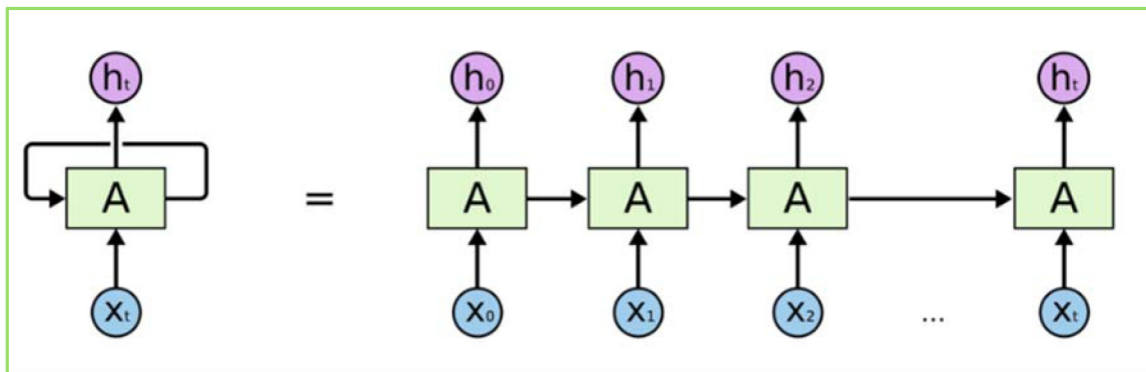


图2-6 RNN 结构特点

由于 RNN 的循环特性，和自然语言文本的序列特性(句子是由单词组成的序列)十分匹配，因此被大量应用于自然语言处理研究中。下图为 RNN 的结构拆解：

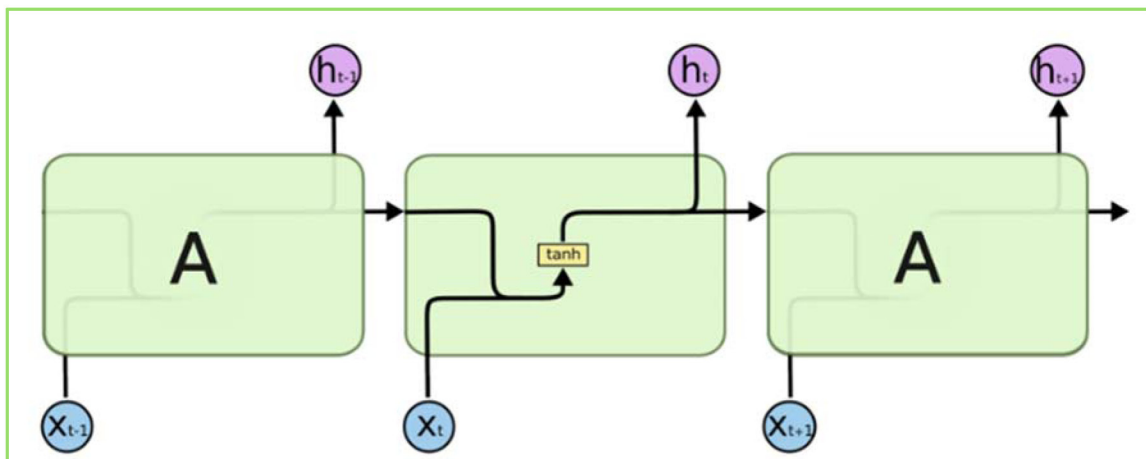


图2-7 RNN 计算单元内部结构

RNN 单个 Cell 的结构简单，因此也造成了梯度消失(Gradient Vanishing)问题，具体表现为 RNN 网络在序列较长时，在序列尾部已经基本丢失了序列首部的信息。为了克服这一问题，LSTM(Long short term memory)被提出，通过门控机制(Gating Mechanism)来控制信息流在每个循环步中的留存和丢弃。下图为 LSTM 的结构拆解：


```

        dropout=dropout,
        batch_first=True)

self.fc = nn.Dense(hidden_dim * 2, output_dim)
self.dropout = nn.Dropout(1 - dropout)
self.sigmoid = ops.Sigmoid()

def construct(self, inputs):
    embedded = self.dropout(self.embedding(inputs))
    _, (hidden, _) = self.rnn(embedded)
    hidden = self.dropout(mnp.concatenate((hidden[-2, :, :], hidden[-1, :, :]), axis=1))
    output = self.fc(hidden)
    return self.sigmoid(output)

```

2.4.4 损失函数与优化器

完成模型主体构建后，首先根据指定的参数实例化网络；然后选择损失函数和优化器，并使用 `nn.TrainOneStepCell` 对其进行封装。针对本节情感分类问题的特性，即预测 Positive 或 Negative 的二分类问题，我们选择 `nn.BCELoss` (二分类交叉熵损失函数)，这里也可以选择 `nn.BCEWithLogitsLoss`，其包含 `sigmoid` 运算，即：

```
BCEWithLogitsLoss = Sigmoid + BCELoss
```

图2-9 损失计算公式

这里使用 `BCELoss` 需要设置 `reduction` 参数为均值。而后使用 `nn.WithLossCell` 将其与实例化网络对象进行关联。

选择合适的损失函数后，选择 Adam 优化器，并将二者传入 `TrainOneStepCell` 中。

```

hidden_size = 256
output_size = 1
num_layers = 2
bidirectional = True
lr = 0.001
pad_idx = vocab.tokens_to_ids('<pad>')

model = RNN(embeddings, hidden_size, output_size, num_layers, bidirectional, pad_idx)
loss_fn = nn.BCEWithLogitsLoss(reduction='mean')
optimizer = nn.Adam(model.trainable_params(), learning_rate=lr)

```

2.4.5 训练逻辑

在完成模型构建，进行训练逻辑的设计。一般训练逻辑分为以下步骤：

- 读取一个 Batch 的数据；
- 送入网络，进行正向计算和反向传播，更新权重；

- 返回 loss。

下面按照此逻辑，使用 tqdm 库，设计训练一个 epoch 的函数，用于训练过程和 loss 的可视化。

```
def forward_fn(data, label):
    logits = model(data)
    loss = loss_fn(logits, label)
    return loss

grad_fn = ms.value_and_grad(forward_fn, None, optimizer.parameters)

def train_step(data, label):
    loss, grads = grad_fn(data, label)
    optimizer(grads)
    return loss

def train_one_epoch(model, train_dataset, epoch=0):
    model.set_train()
    total = train_dataset.get_dataset_size()
    loss_total = 0
    step_total = 0
    with tqdm(total=total) as t:
        t.set_description('Epoch %i' % epoch)
        for i in train_dataset.create_tuple_iterator():
            loss = train_step(*i)
            loss_total += loss.asnumpy()
            step_total += 1
            t.set_postfix(loss=loss_total/step_total)
            t.update(1)
```

2.4.6 评估指标和逻辑

训练逻辑完成后，需要对模型进行评估。即使用模型的预测结果和测试集的正确标签进行对比，求出预测的准确率。由于 IMDB 的情感分类为二分类问题，对预测值直接进行四舍五入即可获得分类标签(0 或 1)，然后判断是否与正确标签相等即可。下面为二分类准确率计算函数实现：

```
def binary_accuracy(preds, y):
    """
    计算每个 batch 的准确率
    """

    # 对预测值进行四舍五入
    rounded_preds = np.around(preds)
    correct = (rounded_preds == y).astype(np.float32)
    acc = correct.sum() / len(correct)
```

```
return acc
```

有了准确率计算函数后，类似于训练逻辑，对评估逻辑进行设计，分别为以下步骤：

- 读取一个 Batch 的数据；
- 送入网络，进行正向计算，获得预测结果；
- 计算准确率。

同训练逻辑一样，使用 tqdm 进行 loss 和过程的可视化。此外返回评估 loss 至供保存模型时作为模型优劣的判断依据。

在进行 evaluate 时，使用的模型是不包含损失函数和优化器的网络主体；在进行 evaluate 前，需要通过 model.set_train(False)将模型置为评估状态，此时 Dropout 不生效。

```
def evaluate(model, test_dataset, criterion, epoch=0):
    total = test_dataset.get_dataset_size()
    epoch_loss = 0
    epoch_acc = 0
    step_total = 0
    model.set_train(False)

    with tqdm(total=total) as t:
        t.set_description('Epoch %i' % epoch)
        for i in test_dataset.create_tuple_iterator():
            predictions = model(i[0])
            loss = criterion(predictions, i[1])
            epoch_loss += loss.asnumpy()

            acc = binary_accuracy(predictions.asnumpy(), i[1].asnumpy())
            epoch_acc += acc

        step_total += 1
        t.set_postfix(loss=epoch_loss/step_total, acc=epoch_acc/step_total)
        t.update(1)

    return epoch_loss / total
```

2.5 模型训练与保存

前序完成了模型构建和训练、评估逻辑的设计，下面进行模型训练。这里我们设置训练轮数为 5 轮。同时维护一个用于保存最优模型的变量 best_valid_loss，根据每一轮评估的 loss 值，取 loss 值最小的轮次，将模型进行保存。

```
num_epochs = 5
best_valid_loss = float('inf')
ckpt_file_name = os.path.join(cache_dir, 'sentiment-analysis.ckpt')
```



```
for epoch in range(num_epochs):
    train_one_epoch(model, imdb_train, epoch)
    valid_loss = evaluate(model, imdb_valid, loss_fn, epoch)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        ms.save_checkpoint(model, ckpt_file_name)
```

训练过程输出如下信息：

```
Epoch 0: 100% | 273/273 [00:48:00:00, 5.59it/s, loss=0.681]
Epoch 0: 100% | 117/117 [00:42:00:00, 2.72it/s, acc=0.581, loss=0.674]
Epoch 1: 100% | 273/273 [00:44:00:00, 6.15it/s, loss=0.661]
Epoch 1: 100% | 117/117 [00:41:00:00, 2.81it/s, acc=0.759, loss=0.519]
Epoch 2: 100% | 273/273 [00:44:00:00, 6.15it/s, loss=0.487]
Epoch 2: 100% | 117/117 [00:41:00:00, 2.82it/s, acc=0.836, loss=0.383]
Epoch 3: 100% | 273/273 [00:44:00:00, 6.15it/s, loss=0.35]
Epoch 3: 100% | 117/117 [00:41:00:00, 2.83it/s, acc=0.868, loss=0.305]
Epoch 4: 100% | 273/273 [00:44:00:00, 6.18it/s, loss=0.298]
Epoch 4: 100% | 117/117 [00:41:00:00, 2.82it/s, acc=0.916, loss=0.219]
```

图2-10 模型训练过程输出信息

2.6 模型加载与测试

模型训练完成后，一般需要对模型进行测试或部署上线，此时需要加载已保存的最优模型(即 checkpoint)，供后续测试使用。这里我们直接使用 MindSpore 提供的 Checkpoint 加载和网络权重加载接口：

- 将保存的模型 Checkpoint 加载到内存中，
- 将 Checkpoint 加载至模型，
- 对测试集按照 batchsize 切分，然后使用 evaluate 方法进行评估，得到模型在测试集上的效果。

```
param_dict = ms.load_checkpoint(ckpt_file_name)
ms.load_param_into_net(model, param_dict)
imdb_test = imdb_test.batch(64)
evaluate(model, imdb_test, loss_fn)
```

输出信息如下：

```
Epoch 0: 100% | 391/391 [00:31:00:00, 12.36it/s, acc=0.873, loss=0.322]

0.32153618827347863
```

图2-11 模型评估过程及输出信息

2.7 自定义输入测试

最后我们设计一个预测函数，实现开头描述的效果，输入一句评价，获得评价的情感分类。具体包含以下步骤：

- 将输入句子进行分词；
- 使用词表获取对应的 index id 序列；
- index id 序列转为 Tensor；
- 送入模型获得预测结果；
- 打印输出预测结果。

具体实现如下

```
score_map = {
    1: "Positive",
    0: "Negative"
}

def predict_sentiment(model, vocab, sentence):
    model.set_train(False)
    tokenized = sentence.lower().split()
    indexed = vocab.tokens_to_ids(tokenized)
    tensor = ms.Tensor(indexed, ms.int32)
    tensor = tensor.expand_dims(0)
    prediction = model(tensor)
    return score_map[int(np.round(ops.sigmoid(prediction).asnumpy()))]
```

最后我们预测开头的样例，可以看到模型可以很好地将评价语句的情感进行分类。

测试样例 1：

```
predict_sentiment(model, vocab, "This film is terrible")
```

输出信息如下：

```
'Negative'
```

测试样例 2：

```
predict_sentiment(model, vocab, "This film is great")
```

输出信息如下：

```
'Positive'
```

2.8 实验小结

本案例通过在 ModelArts 平台创建拥有昇腾 AI 加速芯片的 NoteBook 开发环境，使用 MindSpore 深度学习框架来搭建 RNN 循环神经网络，采用 IMDB 影评数据集进行模型训练、准确率验证，最后我们通过两个测试样例对模型的预测效果进行了演示。通过这个实验，我们可以了解 MindSpore 框架和 ModelArts 平台的使用，了解 RNN 循环神经网络模型的结构和设计

特点，了解词嵌入、经典 RNN 网络结构 LSTM 搭建、门控单元等深度学习自然语言处理技术，了解如何使用一个深度学习模型来解决自然语言处理的具体任务。